Cinema Booking System

Lavet af Aleksandar Neshev & William Holst

Table of Contents

1. Projekt Vision *2*
2. Kravspecifikationer *2*
2.1 Funktionelle krav 2
2.2 Ikke-funktionelle krav 3
3. Use-cases *4*
3.1 Primære aktører 4
3.2 Beskrivelse af brugsscenarier 4
4. Database Design *15*
4.1 Oversigt over enheder og relationer 15
4.2 Detaljer om tabelstruktur 16
4.3 Database forhold 17
5. Analyse af tredje normalform *18*
5.1 Verifikation af tredje normalform 18
5.2 Detaljeret analyse af 3NF-overholdelse 19
5.3 Hvorfor dette design er i tredje normalform 20
5.4 Fordele ved tredje normalform 20
6. Teknisk arkitektur *21*
6.1 Trelagsarkitektur 21
6.2 Autentificering og autorisation 23
6.3 Realtids tilgængelighed af pladser 23
7. Implementeringshøjdepunkter *23*
7.1 Automatisk generering af pladser 23
7.2 Kaskadesletning 23
7.3 Reservationsworkflow 24
7.4 Administrationspanel 24
8. Fremtidige forbedringer *24*
9. Konklusion *25*

1. Projekt Vision

Cinema Booking System er en webbaseret applikation, der er designet til at modernisere og strømline processen med at reservere biografbilletter. Systemet tilbyder en intuitiv platform, hvor brugerne kan gennemse tilgængelige film, vælge spilletider, vælge deres foretrukne pladser og gennemføre reservationer i realtid. Visionen er at skabe en problemfri digital oplevelse, der eliminerer behovet for fysiske billetkøer, samtidig med at biografadministratorer får effektive værktøjer til at administrere deres drift effektivt.

Systemet henvender sig til tre vigtige brugergrupper: faste kunder, der ønsker at booke biografbilletter nemt, biografpersonale, der skal administrere den daglige drift, og administratorer, der har behov for overblik over hele biografdriften, herunder film, spilletider, bookinger og indtægtsopfølgning.

2. Kravspecifikationer

2.1 Funktionelle krav

Brugergodkendelse og -administration

Systemet skal give nye brugere mulighed for at registrere sig med et brugernavn, en e-mailadresse og en adgangskode. Eksisterende brugere skal kunne logge ind sikkert, og systemet skal opretholde brugersessioner under hele deres interaktion. Brugere skal kunne logge ud, når de er færdige. Systemet skal understøtte to forskellige brugerroller: almindelige brugere og administratorer, med forskellige adgangsrettigheder for hver rolle.

Gennemse og vælg film

Brugerne skal kunne se alle aktuelt tilgængelige film i et visuelt tiltalende gitterlayout. Hver filmvisning skal indeholde vigtige oplysninger såsom titel, genre, varighed, vurdering og beskrivelse. Brugerne skal kunne vælge en hvilken som helst film for at se de tilgængelige spilletider.

Valg af spilletid

Når en bruger vælger en film, skal systemet vise alle tilgængelige spilletider for den pågældende film. Hver spilletidsangivelse skal tydeligt vise dato, klokkeslæt, skærmnummer og billetpris. Systemet skal angive, hvor mange pladser der er tilbage til hver forestilling, så brugerne kan træffe informerede beslutninger.

Valg og reservation af pladser

Når brugerne har valgt en spilletid, skal de se en visuel gengivelse af biografens pladsfordeling. Systemet skal tydeligt skelne mellem ledige pladser, allerede reserverede pladser og pladser, der aktuelt er valgt af brugeren. Brugerne skal kunne

vælge flere pladser, dog maksimalt 10 pr. reservation. Systemet skal forhindre dobbeltbookinger ved at sikre, at når en bruger vælger en plads, bliver den utilgængelig for andre brugere i realtid.

Bekræftelse af reservation og historik

Efter valg af pladser skal systemet beregne den samlede pris baseret på antallet af pladser og billetprisen. Brugerne skal kunne bekræfte deres reservation og modtage en øjeblikkelig bekræftelse med alle reservationsoplysninger. Brugerne skal kunne se deres komplette reservationshistorik, herunder både aktive og tidligere reservationer.

Administrative funktioner

Administratorer skal have adgang til et dedikeret adminpanel med omfattende administrationsfunktioner. De skal kunne oprette, redigere og slette film fra systemet. De skal administrere forestillinger, herunder oprette nye forestillinger med specifikke datoer, tidspunkter, priser og salnumre. Administratorer skal kunne se alle bookinger på tværs af alle brugere, annullere bookinger, når det er nødvendigt, og få adgang til statistiske oplysninger, herunder det samlede antal film, forestillinger, bookinger og genererede indtægter.

2.2 Ikke-funktionelle krav

Brugervenlighed

Grænsefladen skal være så intuitiv, at en førstegangsbruger kan gennemføre en reservation uden vejledning. Alle brugerhandlinger skal give klar feedback gennem visuelle indikatorer og bekræftelsesmeddelelser. Fejlmeddelelser skal være nyttige og vejlede brugerne mod korrigerende handlinger.

Ydeevne

Systemet skal indlæse filmoversigter inden for 2 sekunder under normale netværksforhold. Pladsudbuddet skal opdateres i realtid for at undgå bookingkonflikter. Administrationspanelet skal vise statistikker uden mærkbar forsinkelse, selv når der administreres hundredvis af bookinger.

Pålidelighed

Databasen skal til enhver tid opretholde dataintegriteten og sikre, at ingen pladser kan dobbeltbookes, og at alle bookingregistreringer er korrekte. Systemet skal håndtere samtidige brugere på en smidig måde og administrere flere samtidige bookingforsøg uden fejl.

Sikkerhed

Brugeradgangskoder skal opbevares sikkert (selvom den nuværende implementering bruger almindelig tekst som prototype, vil produktionssystemer kræve korrekt hashing). Kun godkendte brugere skal have adgang til reservationsfunktioner, og kun

administratorer skal have adgang til adminfunktioner. Systemet skal forhindre uautoriseret adgang til andre brugeres reservationsoplysninger.

Vedligeholdelse

Kodebasen følger en klar trelagsarkitektur (præsentation, forretningslogik, dataadgang), hvilket gør det nemt at finde og ændre specifikke funktioner. Databasedesignet overholder tredje normalform, hvilket sikrer datakonsistens og nemme opdateringer. Koden indeholder klare kommentarer og følger ensartede navngivningskonventioner.

Skalerbarhed

Databasestrukturen kan rumme vækst i film, spilletider og brugere uden strukturelle ændringer. Den modulære arkitektur gør det muligt at tilføje nye funktioner uden at forstyrre eksisterende funktionalitet.

3. Use-cases

3.1 Primære aktører

Almindelig bruger (kunde)

En person, der besøger biografens websted for at se film og bestille billetter. Dette er den primære brugertype og udgør størstedelen af systeminteraktionerne.
Administrator

En biografmedarbejder med udvidede rettigheder, der administrerer indholdet og driften af reservationssystemet. Administratorer kan udføre alle almindelige brugerhandlinger plus yderligere administrationsfunktioner.

3.2 Beskrivelse af use cases

Brugsscenarie: Opret ny konto

Brugeren navigerer til registreringssiden, indtaster et unikt brugernavn, en gyldig e-mailadresse og en sikker adgangskode (minimum 6 tegn) og bekræfter derefter adgangskoden. Systemet validerer indtastningen, kontrollerer, at brugernavnet ikke allerede findes, opretter den nye brugerkonto og logger automatisk brugeren ind, så vedkommende kan begynde at gennemse film.

Anvendelsestilfælde: Log ind på systemet

En eksisterende bruger indtaster sit brugernavn og sin adgangskode på login-siden. Systemet verificerer disse legitimationsoplysninger i forhold til databasen. Hvis de er korrekte, opretter det en session for brugeren og viser siden med filmudvalg. Hvis brugeren er administrator, får vedkommende adgang til yderligere administratorfunktioner.

Brugsscenarie: Gennemse tilgængelige film

Når brugeren er logget ind, ser vedkommende et oversigtsbillede over alle aktuelt tilgængelige film. Hvert filmkort viser vigtige oplysninger, herunder titel, genre, varighed, vurdering og en kort beskrivelse. Brugeren kan klikke på en film for at gå videre til valg af spilletid.

Anvendelsestilfælde: Vælg spilletid

Efter at have valgt en film, ser brugeren alle planlagte spilletider for den pågældende film. Hver spilletid viser dato, klokkeslæt, skærmnummer, billetpris og antal ledige pladser. Brugeren vælger den ønskede spilletid ved at klikke på den, hvorefter vedkommende føres til grænsefladen for valg af pladser.

Anvendelsestilfælde: Vælg pladser og afslut booking

Brugeren ser et visuelt sædeoversigt, der viser biografens layout. Grønne pladser er ledige, grå pladser er allerede reserveret, og røde pladser er i øjeblikket valgt af brugeren. Brugeren klikker på ledige pladser for at vælge dem (op til 10 pladser maksimalt). Systemet viser en løbende oversigt med de valgte pladsenumre, antal og samlet pris. Når brugeren er tilfreds med sit valg, klikker vedkommende på knappen "Bekræft booking". Systemet behandler bookingen, markerer de valgte pladser som utilgængelige og viser en bekræftelsesmeddelelse med alle bookingoplysninger, herunder bekræftelsesnummer.

Anvendelsestilfælde: Se bookinghistorik

Fra hovedmenuen kan brugeren få adgang til sin bookinghistorik. Systemet viser alle tidligere og aktuelle bookinger med filmtitel, spilletid, bookede pladser, samlet pris og bookingstatus (bekræftet eller annulleret). Dette giver brugerne mulighed for at se deres kommende film eller gennemgå deres bookingmønstre.

Anvendelsestilfælde: Administrer film (kun administrator)

En administrator åbner adminpanelet og navigerer til afsnittet om filmadministration. Her kan de tilføje nye film ved at indtaste titel, beskrivelse, genre, varighed og vurdering. De kan redigere eksisterende film for at opdatere oplysninger. De kan også slette film, men dette vil medføre, at alle tilknyttede spilletider og bookinger slettes.

Anvendelsestilfælde: Administrer spilletider (kun administrator)

I adminpanelets sektion for visningstider kan administratorer oprette nye visningstider ved at vælge en film, angive skærmnummer, vælge dato og klokkeslæt, indstille billetprisen og definere det samlede antal pladser (som standard er 80). Systemet genererer automatisk sædeindretningen. Administratorer kan redigere eksisterende visningstider for at justere tider eller priser eller slette visningstider, der ikke længere er nødvendige.

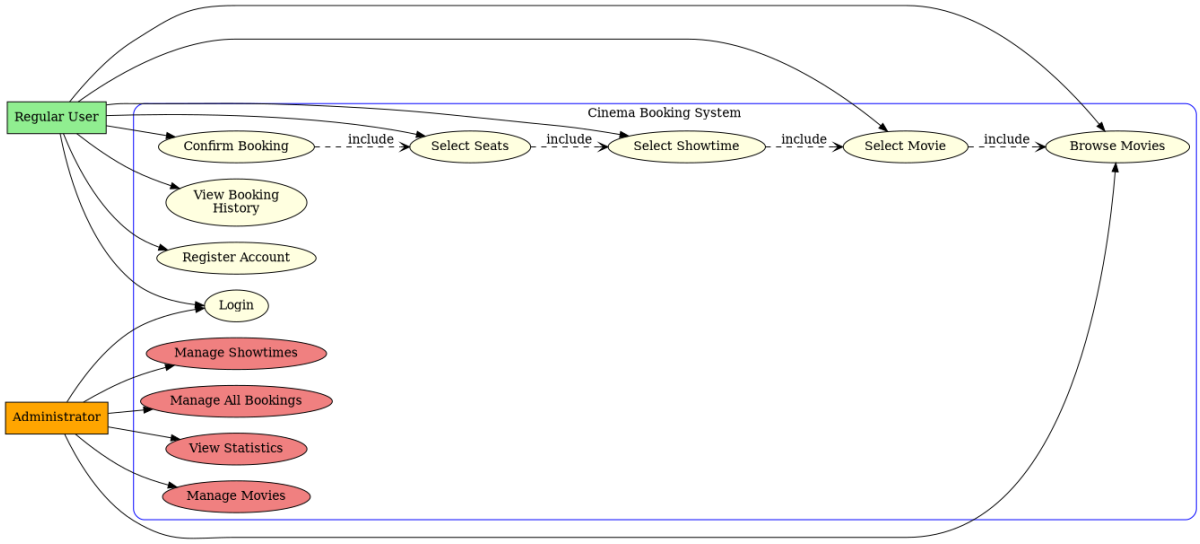
Anvendelsestilfælde: Administrer bookinger (kun administrator)

Administratorer kan se alle bookinger for alle brugere i en overskuelig tabel, der viser brugernavn, film, spilletid, pris og status. De kan annullere bekræftede bookinger, hvis

det er nødvendigt (f.eks. hvis en forestilling aflyses), hvilket frigiver pladserne igen. De kan også slette annullerede eller ventende bookinger permanent for at rydde op i databasen.

Anvendelsestilfælde: Se statistik (kun administrator)

Administratorer har adgang til statistikdashboardet, der viser nøgletal, herunder det samlede antal film i systemet, det samlede antal aktive forestillinger, det samlede antal bekræftede bookinger og den samlede indtægt fra alle bekræftede bookinger. Dette giver et hurtigt overblik over biografens driftsstatus og økonomiske resultater.



3.3 Use-Cases forklarede

Use Case 1: Register et ny account

Når en ny bruger ønsker at oprette en konto, udfylder vedkommende en registreringsformular med brugernavn, e-mailadresse og adgangskode. Systemet kontrollerer først, om brugernavnet allerede findes for at undgå dubletter, og opretter derefter den nye brugerpost i databasen.

Først, tjek hvis brugernavnet eksisterer

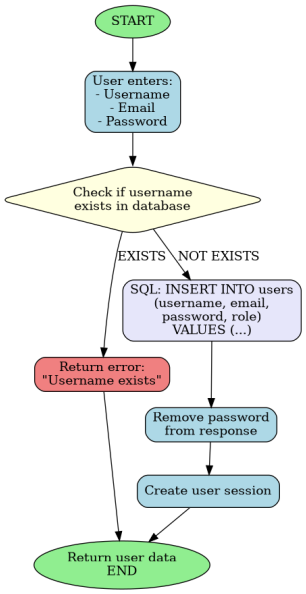
```
const { data, error } = await supabase
    .from('users')
    .select('*')
    .eq('username', userData.username)
    .maybeSingle();
```

Metoden .eq() filtrerer efter matchende brugernavn, og .maybeSingle() returnerer enten brugeren eller null (giver ikke fejl, hvis den ikke findes).

Vi skal nu skabe en ny user.

```
const { data, error } = await supabase
    .from('users')
    .insert({
        username: userData.username,
        email: userData.email,
        password: userData.password,
        role: 'user'
    })
    .select();
```

Metoden .insert() tilføjer den nye post, og .select() returnerer straks de nyoprettede brugerdata (svarende til SQL's RETURNING-klausul).



Use Case 2: Login ind i systemet

Brugeren indtaster sit brugernavn og sin adgangskode. Systemet søger i databasen efter en brugerpost, der matcher både brugernavnet OG adgangskoden. Hvis der findes en match, logges brugeren ind, og der oprettes en session.

```
const { data, error } = await supabase
    .from('users')
    .select('*')
    .eq('username', username)
    .eq('password', password)
    .single();
```

Det kaldes for "authentication query". Denne query kæder to .eq()-metoder sammen for at filtrere efter både brugernavn OG adgangskode. Metoden .single() forventer nøjagtigt et resultat: den returnerer brugerobjektet, hvis det findes, eller genererer en fejl, hvis der ikke findes noget match.

Use Case 3: Søg for filmer

Når en bruger logger ind eller navigerer til filmsiden, indlæser systemet alle tilgængelige film fra databasen og viser dem i et gitterformat. Hver film viser sin titel, genre, varighed, vurdering og beskrivelse.

```
const { data, error } = await supabase
    .from('movies')
    .select('*')
    .order('title');
```

Metoden .order('title') sorterer resultaterne alfabetisk efter titlen, hvilket gør det nemmere for brugerne at gennemse film.

Use Case 4: Vælge showtime og se fri sæder

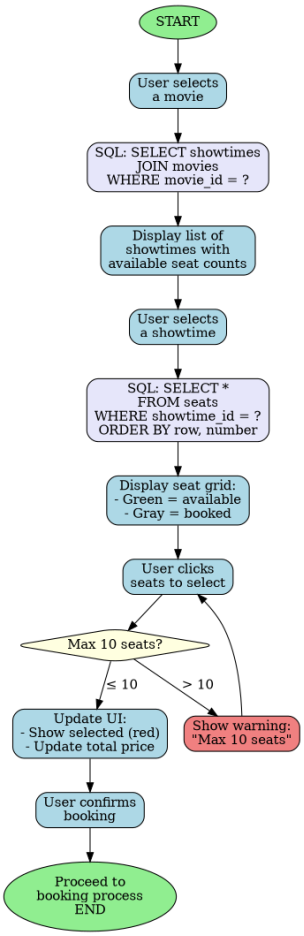
Når en bruger vælger en film, indlæser systemet alle visningstider for den pågældende film. Når en bruger vælger en bestemt visningstid, indlæser systemet visningstidsoplysningerne sammen med alle pladser og viser, hvilke der er ledige, og hvilke der allerede er reserveret.

```
const { data, error } = await supabase
    .from('showtimes')
    .select(`
        *,
        movies(title),
        seats(id, is_available)
    `)
    .eq('movie_id', movieId)
    .order('start_time');
```

Denne forespørgsel bruger Supabases relationssyntaks. Den indlejrede movies(title) henter den relaterede filmtitel, og seats(id, is_available) henter alle pladser med deres tilgængelighedsstatus. Backticks tillader flerstrengede strenge for læsbarhed.

```
const { data, error } = await supabase
    .from('seats')
    .select('*')
    .eq('showtime_id', showtimeId)
    .order('row_letter')
    .order('seat_number');
```

Dette returnerer alle 80 pladser til forestillingen. De to .order()-metoder kædes sammen for først at sortere efter rækkebogstav (A, B, C...), derefter efter pladsnummer (1, 2, 3...) inden for hver række.



Use Case 5: Booking

Dette er den mest komplekse use case, der involverer flere databaseoperationer. Brugeren vælger pladser, bekræfter reservationen, og systemet skal oprette en reservationspost, knytte pladserne til reservationen, markere pladserne som utilgængelige og beregne den samlede pris.

```
const { data, error } = await supabase
    .from('bookings')
    .insert({
        user_id: userId,
        showtime_id: showtimeId,
        booking_date: new Date().toISOString(),
        status: 'pending'
    })
    .select();
```

.select() returnerer det nyoprettede bookingobjekt inklusive dets automatisk genererede ID.

```
const { data, error } = await supabase
    .from('seats')
    .select('*')
    .eq('id', seatId)
    .single();
```

Her tjekker vi hvis denne seat er taget.

```
const { data, error } = await supabase
    .from('seats')
    .update({ is_available: false })
    .eq('id', seatId)
    .select();
```

Her markerer vi denne seat som unavailable (is_available = false).

Metoden .update() ændrer posten, og .eq('id', seatId) sikrer, at kun denne specifikke plads opdateres.

```
const { data, error } = await supabase
    .from('booked_seats')
    .insert({
        booking_id: bookingId,
        seat_id: seatId
    })
    .select();
```

Her binder vi denne seat til booking.

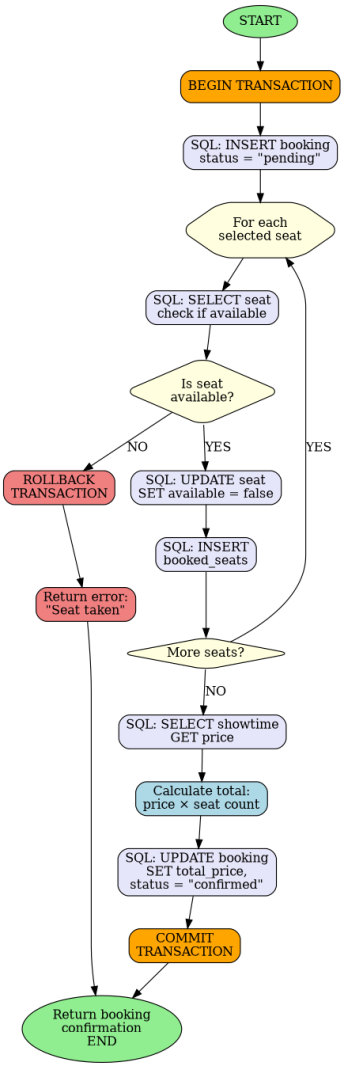
```
const { data, error } = await supabase
    .from('showtimes')
    .select('price')
    .eq('id', showtimeId)
    .single();
```

Her får vi prisen vi skal betale.

```
const { data, error } = await supabase
    .from('bookings')
    .update({
        total_price: totalPrice,
        status: 'confirmed'
    })
    .eq('id', bookingId)
    .select();
```

Here we opdaterer booking-processen med den finale pris og status.

Denne use case udfører flere databaseoperationer, som alle skal lykkes eller alle mislykkes sammen. Selvom Supabase ikke eksponerer eksplicit transaktionskontrol i JavaScript-klienten, sikrer PostgreSQL-databasen nedenunder atomicitet. I produktion vil kritiske flerstrengede operationer som denne bruge databasefunktioner eller PostgreSQL RPC-funktionen til at garantere transaktionsintegritet.



Use Case 6: Se Booking History

Brugeren anmoder om at se alle sine tidligere og aktuelle bookinger. Systemet søger i databasen efter alle bookinger, der tilhører denne bruger, og sammenkæder dem med spilletider og film for at vise komplette oplysninger.

```
const { data, error } = await supabase
    .from('bookings')
    .select(`
        *,
        showtimes(
            *,
            movies(title)
        )
    `)
    .eq('user_id', userId)
    .order('booking_date', { ascending: false });
```

Her får vi brugerens booking history

Dette bruger indlejrede relationsforespørgsler. Showtimes(*) henter alle showtime-data, og inden for disse henter movies(title) filmtitlen. .order() med { ascending: false } sorterer de nyeste bookinger først.

```
const { data, error } = await supabase
    .from('booked_seats')
    .select('*')
    .eq('booking_id', bookingId);
```

Her henter vi alle seats til hver booking

Denne query udføres typisk flere gange (én gang pr. booking) for at få oplysninger om pladserne for hver reservation.

Use Case 7: Opret Showtime med automatisk sædegenerering (admin)

Når en administrator opretter en ny forestilling, angiver vedkommende film, tidspunkt, sal og pris. Systemet genererer automatisk 80 pladser (8 rækker × 10 pladser) til den pågældende forestilling, hvilket eliminerer behovet for manuel oprettelse af pladser.

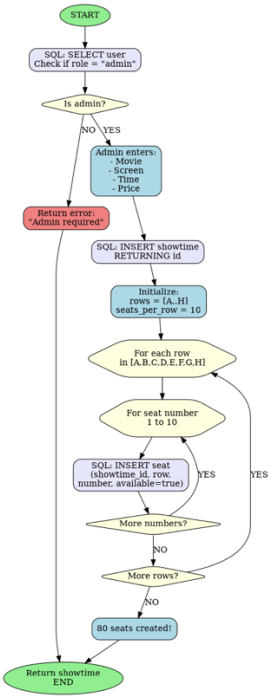
```
const { data, error } = await supabase
    .from('showtimes')
    .insert({
        movie_id: showtimeData.movie_id,
        screen_number: showtimeData.screen_number,
        start_time: showtimeData.start_time,
        price: showtimeData.price
    })
    .select();
```

Her skaber vi en "showtime"

De returnerede data indeholder ID'et for den nye forestilling, som er nødvendigt for at oprette pladser.

```
const { data, error } = await supabase
    .from('seats')
    .insert({
        showtime_id: showtimeId,
        seat_number: seatNumber,
        row_letter: rowLetter,
        is_available: true
    })
    .select();
```

Her genererer vi et plads (seat). Det er i en for-løkke typisk til 80 sæder.



Use Case 8: Annuller Booking (admin eller bruger)

Når en reservation skal annulleres, skal systemet frigive alle pladserne til tilgængelig status og opdatere reservationsstatus. Dette sikrer, at disse pladser kan reserveres af andre brugere.

```
const { data, error } = await supabase
    .from('bookings')
    .select('*')
    .eq('id', bookingId)
    .single();
```

Her henter vi booking details.

```
const { data, error } = await supabase
    .from('booked_seats')
    .select('*')
    .eq('booking_id', bookingId);
```

Her henter vi alle seats til denne booking.

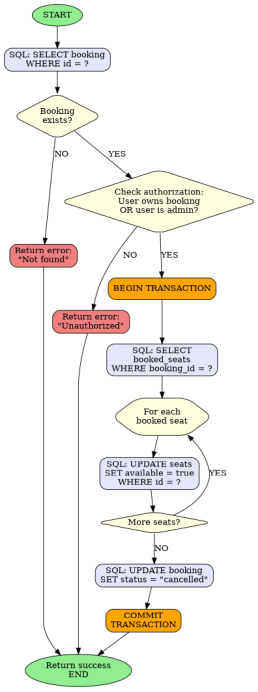
Dette returnerer en matrix med alle booked_seats-poster for denne reservation.

```
const { data, error } = await supabase
    .from('seats')
    .update({ is_available: true })
    .eq('id', seatId)
    .select();
```

Dette kaldes i en løkke for hvert sæde for at markere dem som ledige igen.

```
const { data, error } = await supabase
    .from('bookings')
    .update({ status: 'cancelled' })
    .eq('id', bookingId)
    .select();
```

Og til sidst, opdatere booking-statusen.



Use Case 9: View Statistics (admin)

Administratorer kan se nøgletal om biografens drift. Systemet udfører flere forespørgsler for at tælle poster og sammenlægge indtægter.

```
const { count, error } = await supabase
    .from('movies')
    .select('*', { count: 'exact', head: true });
```

Tæl det samlede antal film.

Indstillingen { count: 'exact', head: true } fortæller Supabase, at den kun skal returnere antallet, ikke de faktiske data, hvilket gør det hurtigere.

```
const { count, error } = await supabase
    .from('showtimes')
    .select('*', { count: 'exact', head: true });
```
Tæl det samlede antal showtimes.

```
const { count, error } = await supabase
    .from('bookings')
    .select('*', { count: 'exact', head: true })
    .eq('status', 'confirmed');
```
Tæl bekræftede bookinger.

.eq('status', 'confirmed') filtrerer, så kun bookinger med bekræftet status tælles med.

```
const { data, error } = await supabase
    .from('bookings')
    .select('total_price')
    .eq('status', 'confirmed');

// Then calculate sum in JavaScript
const totalRevenue = data.reduce((sum, booking) =>
    sum + (booking.total_price || 0), 0
);
```
Beregn samlet omsætning.

Supabase har ikke en direkte SUM-funktion i JavaScript-klienten, så vi henter alle total_price-værdierne og summerer dem ved hjælp af JavaScripts reduce()-metode.

4. Database Design

4.1 Oversigt over entity-relationship

Databasen består af syv hovedtabeller, der arbejder sammen om at administrere hele arbejdsgangen for biografreservationer. Disse tabeller repræsenterer de centrale enheder i systemet og deres relationer.

"user"-tabellen gemmer alle brugerkonti med autentificeringsoplysninger og rolleoplysninger. Hver bruger kan have flere reservationer, hvilket skaber en en-til-mange-relation mellem brugere og reservationer.

"movies"-tabellen indeholder oplysninger om alle film, der er tilgængelige for visning. Hver film kan have flere planlagte visningstider, hvilket skaber en en-til-mange-relation mellem film og visningstider.

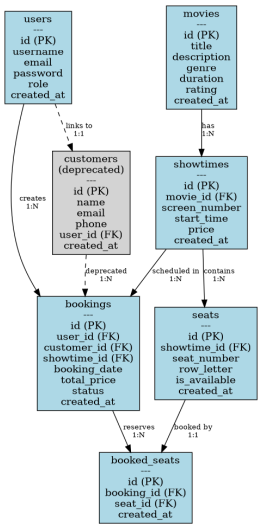
"showtimes"-tabellen repræsenterer specifikke filmvisninger på bestemte tidspunkter og skærme. Hver visningstid tilhører nøjagtigt én film, men kan have mange pladser, hvilket etablerer en en-til-mange-relation med pladstabellen.

"seats"-tabellen repræsenterer individuelle pladser for hver visningstid. Hver plads tilhører én visningstid og kan bookes af højst én booking, men kan forblive ubooket.

"bookings"-tabellen registrerer reservationstransaktioner foretaget af brugere. Hver booking oprettes af én bruger til én bestemt visningstid, men kan omfatte flere pladser.

"booked_seats"-tabellen er en krydstabell, der implementerer mange-til-mange-forholdet mellem bookinger og pladser. Dette gør det muligt at reservere flere pladser i en booking og spore, hvilke specifikke pladser der hører til hvilken booking.

Tabellen "customers" forbliver i skemaet af hensyn til bagudkompatibilitet, men er udfaset til fordel for tabellen "users". I tidligere versioner blev kundeoplysninger gemt separat fra brugergodkendelse.



4.2 Detaljer om tabelstruktur

Tabellen "users" administrerer brugergodkendelse og -autorisation. Feltet id er en seriel primærnøgle, der entydigt identificerer hver bruger. Feltet username skal være unikt og gemmer det login-navn, som brugeren har valgt. Feltet email skal også være unikt og gemmer brugerens e-mail-adresse. Feltet password gemmer godkendelsesoplysningerne (i produktionen skal disse hashes, men prototypen bruger almindelig tekst). Feltet role er begrænset til enten "user" eller "admin" med standardindstillingen "user", som bestemmer adgangsniveauerne i hele applikationen. Tidsstemplet "created_at" registrerer, hvornår kontoen blev oprettet.

Tabellen "movies" gemmer biografens filmkatalog. Id-feltet er en seriel primærnøgle. Titel-feltet gemmer filmens navn og er obligatorisk. Beskrivelsesfeltet giver en tekstoversigt over handlingen eller indholdet. Genre-feltet kategoriserer filmen (action, drama, sci-fi osv.). Feltet duration gemmer filmens længde i minutter som et heltal. Feltet "rating" angiver alders-/indholdsklassificeringen, f.eks. PG-13 eller R. Tidsstemplet "created_at" markerer, hvornår filmen blev tilføjet til systemet.

Tabellen "showtimes" repræsenterer planlagte filmvisninger. Feltet "id" er en seriel primærnøgle. Feltet "movie_id" er en fremmednøgle, der henviser til tabellen movies og

angiver, hvilken film der vises. Feltet "screen_number" angiver, hvilken biograf eller hvilken sal filmen vises i. Feltet start_time er et tidsstempel, der angiver nøjagtigt, hvornår visningen begynder. Feltet price gemmer billetprisen som et decimaltal med to decimaler. Tidsstemplet "created_at" registrerer, hvornår denne visningstid blev planlagt. Når en film slettes, slettes alle dens visningstider automatisk på grund af ON DELETE CASCADE-begrænsningen.

Tabellen "seats" definerer antallet af pladser for hver visningstid. Feltet "id" er en seriel primærnøgle. Feltet "showtime_id" er en fremmednøgle, der henviser til tabellen showtimes og knytter hver plads til en bestemt visning. Feltet "seat_number" gemmer den numeriske position inden for en række (1, 2, 3 osv.). Feltet "row_letter" gemmer rækkeidentifikatoren (A, B, C osv.). Det boolske felt "is_available" sporer, om pladsen stadig kan reserveres. Tidsstemplet "created_at" registrerer, hvornår pladsen blev genereret. En unik begrænsning på (showtime_id, row_letter, seat_number) sikrer, at der ikke findes dublerede pladser for nogen forestilling. Når en forestilling slettes, slettes alle dens pladser automatisk via CASCADE.

Tabellen "bookings" registrerer reservationstransaktioner. Feltet "id" er en seriel primærnøgle. Feltet "user_id" er en fremmednøgle til tabellen users, der identificerer, hvem der har foretaget reservationen. Feltet "customer_id" er en valgfri fremmednøgle til den udfasede tabel customers for bagudkompatibilitet. Feltet "showtime_id" er en fremmednøgle til showtimes, der angiver, hvilken forestilling der er reserveret. Tidsstemplet "booking_date" registrerer, hvornår reservationen blev oprettet. Det decimale felt "total_price" gemmer den endelige pris for alle pladser i denne reservation. Feltet status er begrænset til "pending" (afventende), "confirmed" (bekræftet) eller "cancelled" (annulleret) og er som standard "pending". Tidsstemplet "created_at" markerer tidspunktet for oprettelsen af posten. CASCADE-sletninger sikrer, at hvis en bruger eller en forestilling slettes, fjernes de tilknyttede reservationer.
"customers" tabel (udgået)
Denne tabel blev brugt i tidligere versioner, før brugergodkendelse blev implementeret. Den gemmer kundekontaktoplysninger separat fra brugerkonti. Selvom den stadig findes af kompatibilitetshensyn, bruger nye funktioner udelukkende brugertabellen.

4.3 Database forhold

Databasen implementerer flere vigtige forhold, der håndhæver forretningsregler og dataintegritet.

Forholdet mellem brugere og bookinger er en-til-mange. En enkelt bruger kan oprette flere bookinger over tid, men hver booking tilhører nøjagtigt én bruger. Dette håndhæves gennem fremmednøglen user_id i bookingstabellen.

Forholdet mellem film og visningstider er en-til-mange. En film kan have flere planlagte visningstider (forskellige dage, tidspunkter, skærme), men hver visningstid viser nøjagtigt én film. Dette styres gennem den fremmede nøgle movie_id i showtimes.

Forholdet mellem forestillinger og pladser er en-til-mange. Hver forestilling har sit eget komplette sæt pladser (typisk 80 pladser arrangeret i rækker), men hver plads tilhører nøjagtigt én forestilling. Den fremmede nøgle showtime_id i pladser opretholder dette forhold.

Forholdet mellem showtimes og bookings er en-til-mange. Flere brugere kan booke forskellige pladser til samme showtime, hvilket skaber flere bookingposter. Hver booking gælder dog kun for én bestemt showtime. Dette håndhæves af den fremmede nøgle showtime_id i bookings.

Forholdet mellem bookinger og pladser er mange-til-mange, implementeret gennem forbindelsestabellen booked_seats. En booking kan omfatte flere pladser (op til 10), og teoretisk set kan en plads være en del af flere bookinger i løbet af sin levetid (selvom forretningslogikken forhindrer dobbeltbookinger i øjeblikket).

Denne fleksible struktur gør det muligt for systemet at spore nøjagtigt, hvilke pladser der hører til hvilken booking.

5. Analyse af tredje normalform

Databasenormalisering er processen med at organisere data for at reducere redundans og forbedre integriteten. Tredje normalform (3NF) er et normaliseringsniveau, der sikrer, at data er velstrukturerede og effektive.

5.1 Verifikation af tredje normalform

For at være i tredje normalform skal en database opfylde tre betingelser: Den skal være i første normalform (1NF), anden normalform (2NF) og tredje normalform (3NF) selv.

Krav til første normalform (1NF)

En tabel er i 1NF, hvis den kun indeholder atomare værdier (ingen gentagende grupper eller arrays) og hver række er unik. Alle tabeller i biografbookingsystemet opfylder dette krav. Brugerbordet gemmer enkeltværdier i hvert felt (brugernavn, e-mail, adgangskode) uden arrays eller indlejrede data. Filmtabellen gemmer på samme måde atomare værdier for titel, beskrivelse, genre, varighed og vurdering. Tabellen med pladser gemmer individuelle pladsoplysninger (seat_number, row_letter) som separate atomare værdier i stedet for arrays af pladser. Hver tabel har en primær nøgle, der sikrer, at hver række er unik.

Krav til anden normalform (2NF)

En tabel er i 2NF, hvis den er i 1NF, og alle ikke-nøgleattributter er fuldt afhængige af hele primærnøglen. Dette er primært relevant for tabeller med sammensatte primærnøgler. Biografreserveringsdatabasen opfylder 2NF, fordi tabellerne bruger enkeltkolonne-surrogatnøgler (serielle id-felter) i stedet for sammensatte nøgler, så der ikke er nogen delvise afhængigheder. I tabellen med pladser afhænger alle attributter (showtime_id, seat_number, row_letter, is_available) for eksempel af den komplette primære nøgle (id) og ikke kun en del af den.

Krav til tredje normalform (3NF)

En tabel er i 3NF, hvis den er i 2NF, og ingen ikke-nøgleattributter afhænger af et andet ikke-nøgleattribut (ingen transitive afhængigheder). Biografreserveringsdatabasen opnår dette gennem korrekt relationsadskillelse.

5.2 Detaljeret analyse af 3NF-overholdelse

user-tabel

Alle attributter (brugernavn, e-mail, adgangskode, rolle) afhænger direkte af primærnøglen (id) og ikke af hinanden. Der er ingen transitive afhængigheder. For eksempel bestemmer rollen ikke e-mailen, og brugernavnet bestemmer ikke adgangskoden i en funktionel afhængighedsforstand. Denne tabel er i 3NF.

movies-tabel

Hvert attribut (titel, beskrivelse, genre, varighed, vurdering) afhænger udelukkende af film-id'et. Genren bestemmer ikke varigheden, og titlen bestemmer ikke vurderingen. Alle data er specifikke for den film, der identificeres ved primærnøglen. Denne tabel er i 3NF.

showtimes-tabellen

Attributterne for visningstid (film-id, skærmnummer, starttid, pris) afhænger alle af visningstids-id'et. Det er vigtigt at bemærke, at prisen gemmes på visningstidsniveauet snarere end på filmniveauet, da priserne kan variere afhængigt af tidspunktet på dagen, ugedagen eller særlige begivenheder. Dette forhindrer en transitiv afhængighed, hvor prisen ville afhænge af film-id'et gennem visningstids-id'et. Hver visningstid har sin egen uafhængige pris. Denne tabel er i 3NF.

seats-tabellen

Alle sædeattributter (showtime_id, seat_number, row_letter, is_available) afhænger af sæde-id'et. Tilgængelighedsstatus er specifik for hvert enkelt sæde og bestemmes ikke af række eller nummer. Sædets placering (række og nummer) bestemmer ikke tilgængeligheden. Denne tabel er i 3NF.

bookings-tabellen

Bookingsattributterne (user_id, showtime_id, booking_date, total_price, status) afhænger af booking-id'et. Total_price beregnes ud fra bookingens pladser og

showtime-prisen, men gemmes her som en denormaliseret cache af hensyn til ydeevne og historisk nøjagtighed (hvis priserne ændres senere). Dette er en acceptabel undtagelse fra streng 3NF af praktiske årsager, og forretningslogikken sikrer konsistens. Status afhænger af selve bookingen, ikke af brugeren eller showtime. Denne tabel er funktionelt i 3NF med praktisk optimering.

booked_seats-tabel

Denne krydstabeller indeholder kun fremmednøgler (booking_id, seat_id), der tilsammen repræsenterer forholdet. Der er ingen ikke-nøgleattributter, der kan skabe transitive afhængigheder. Denne tabel er i 3NF.

5.3 Hvorfor dette design er i tredje normalform

Databasen opnår tredje normalform gennem omhyggelig entitetsseparation og relationsdesign. Hver tabel repræsenterer et enkelt, veldefineret koncept med attributter, der kun afhænger af det pågældende koncepts identifikator.

Brugeroplysninger er adskilt fra bookingoplysninger. Brugerattributter som brugernavn og e-mail gemmes i brugertabellen, mens bookingspecifikke attributter som bookingdato og samlet pris gemmes i bookinger. Denne adskillelse forhindrer redundans – hvis vi gemte brugernavn og e-mail med hver booking, ville ændring af en brugers e-mail kræve opdatering af flere bookingposter. I stedet refererer bookingen blot til brugeren via user_id.

Filminformation er adskilt fra visningstidsoplysninger. Filmattributter som titel og genre gemmes én gang i filmtabellen. Visningstidsspecifikke attributter som start_time og screen_number findes i visningstidstabellen, hvor de henviser til filmen. Det betyder, at vi kan have flere visningstider for den samme film uden at duplikere filmbeskrivelsen i hver visningstidsrekord.

Sædeoplysninger er adskilt fra visningstidsoplysninger. I stedet for at gemme 80 sæder som en matrix eller gentagne kolonner i visningstidstabellen, er hvert sæde en separat rekord. Dette gør det muligt for sæder at have deres egen tilgængelighedsstatus og blive refereret til individuelt ved bookinger.

Forholdet mellem booking og sæde bruger en krydstabell. I stedet for at gemme kommaseparerede sæde-id'er i bookingposten (hvilket ville være i strid med 1NF) eller gemme booking-id'et flere gange i sædetabellen (hvilket ville forhindre sæder i at blive genanvendt), opretter tabellen booked_seats et rent mange-til-mange-forhold.

5.4 Fordele ved tredje normalform

Denne normaliserede struktur giver flere vigtige fordele. Datakonsistensen opretholdes, fordi hver enkelt oplysning findes på nøjagtig ét sted. Hvis en filmtitel skal rettes, ændres den én gang i filmtabellen og afspejles automatisk i alle forestillinger.

Opdateringsanomalier undgås – du kan ikke ved et uheld opdatere en filmtitel i én forestilling og glemme at opdatere den i en anden, fordi titlen slet ikke er gemt i forestillingstabellen.

Sletningsanomalier undgås gennem korrekt kaskadering. Når en film slettes, fjernes dens visningstider automatisk, hvilket udløser fjernelsen af deres pladser, hvilket udløser fjernelsen af alle bookinger og reserverede pladser. Dette opretholder referentiel integritet uden forældreløse poster.

Designet er fleksibelt og skalerbart. Tilføjelse af en ny attribut til film (f.eks. instruktørens navn) kræver kun tilføjelse af en kolonne til en tabel. Tilføjelse af nye visningstider for eksisterende film kræver ingen ændringer i filmtabellen. Strukturen tilpasser sig naturligt til vækst og forandringer.

Lagringseffektiviteten optimeres ved at fjerne redundans. Filmbeskrivelser og genrer gemmes én gang og gentages ikke i hver visningstid eller reservationspost. Dette reducerer databasestørrelsen og forbedrer ydeevnen.

6. Teknisk arkitektur

6.1 Trelagsarkitektur

Biografreserveringssystemet implementerer en ren trelagsarkitektur, der adskiller opgaver og fremmer vedligeholdelsen. Dette arkitekturmønster opdeler applikationen i præsentations-, forretningslogik- og dataadgangslag.

Præsentationslag (klientside)

Præsentationslaget består af HTML-, CSS- og JavaScript-filer, der kører i brugerens browser. Filen index.html definerer sidestrukturen og indeholder alle UI-sektioner (godkendelse, film, spilletider, sædevalg, bekræftelse, historik, adminpanel). Filen style.css leverer al visuel styling, herunder gitterlayouter, farveskemaer, responsivt design og animationer. Filen app.js indeholder klient-side logik, der håndterer brugerinteraktioner, foretager API-kald til serveren og opdaterer grænsefladen dynamisk.

Dette lag er kun ansvarligt for at vise information og registrere brugerinput. Det indeholder ikke forretningslogik eller direkte databaseadgang. Når en brugerhandling kræver data, sender præsentationslaget en HTTP-anmodning til forretningslogiklaget og opdaterer visningen, når svaret ankommer.

Forretningslogiklag (servicelag)

Forretningslogiklaget er implementeret i cinema-server.js og cinemaServicelag.js. Filen cinema-server.js konfigurerer Express-webserveren og definerer alle API-endpoints

(ruter), som klienten kan kalde. Den håndterer HTTP-anmodninger, validerer, at brugerne er autentificerede og autoriserede, kalder de relevante servicefunktioner og sender svar tilbage til klienten.

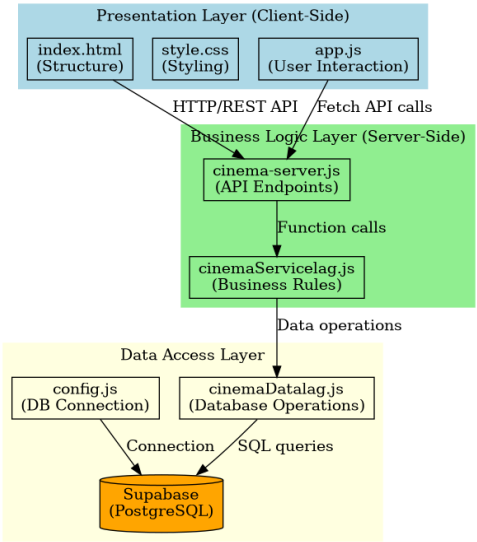
Filen cinemaServicelag.js indeholder den centrale forretningslogik – de regler, der styrer, hvordan biografbookingsystemet fungerer. Dette lag afgør, om en bruger kan udføre en handling (f.eks. kontrollere, om en bruger er administrator, før vedkommende får lov til at slette en film), koordinerer komplekse operationer (f.eks. oprette en forestillingstid og automatisk generere 80 pladser til den) og omdanner data mellem det format, klienten forventer, og det format, databasen gemmer.

Denne adskillelse betyder, at forretningsreglerne findes ét sted. Hvis vi har brug for at ændre, hvordan sædevalg fungerer (f.eks. ved at begrænse bookinger til 5 pladser i stedet for 10), ændrer vi servicelaget uden at røre ved databasefunktionerne eller klientgrænsefladen.

Dataadgangslag (datalag)

Dataadgangslaget er implementeret i cinemaDatalag.js og config.js. Filen config.js opretter forbindelsen til Supabase-databasen ved hjælp af miljøvariabler. Filen cinemaDatalag.js indeholder alle funktioner, der interagerer direkte med databasen via SQL-forespørgsler. Hver funktion udfører en specifik databaseoperation – henter film, opretter en reservation, opdaterer sæde tilgængelighed osv.

Dette lag ved intet om forretningsregler eller brugergrænseflader. Det udfører blot databaseoperationer og returnerer resultater. Hvis vi beslutter at skifte fra Supabase til PostgreSQL eller MySQL, behøver vi kun at ændre dette lag, mens forretningslogikken og præsentationen forbliver uændret.



6.2 Autentificering og autorisation

Systemet implementerer sessionsbaseret autentificering og rollebaseret autorisation. Når en bruger logger ind, validerer serveren deres legitimationsoplysninger i forhold til databasen og opretter en session i hukommelsen (i et produktionssystem ville dette bruge en passende sessionslager). Serveren returnerer brugerobjektet, inklusive deres rolle, til klienten.

Ved efterfølgende anmodninger inkluderer klienten bruger-id'et i X-User-Id-headeren. Serveren bruger middleware-funktionerne requireAuth og requireAdmin til at verificere, at brugeren er logget ind og har de nødvendige tilladelser, før anmodninger behandles. Dette forhindrer uautoriseret adgang til reservationsfunktioner og admin-funktioner.

6.3 Realtids tilgængelighed af pladser

Når en bruger vælger en forestillingstid og viser grænsefladen til valg af pladser, indlæser systemet alle pladser til den pågældende forestillingstid fra databasen. Ledige pladser vises i grønt, mens reserverede pladser vises i gråt. Når brugeren klikker på pladser for at vælge dem, opdateres grænsefladen straks, så de vises i rødt.

Når brugeren bekræfter reservationen, markerer systemet de valgte pladser som utilgængelige i databasen inden for en transaktion, hvilket sikrer, at hvis flere brugere forsøger at reservere den samme plads samtidigt, vil kun én af dem lykkes. Transaktionen opretholder is_available-flagget på hver pladsregistrering, hvilket forhindrer dobbeltbooking.

7. Implementeringshøjdepunkter

7.1 Automatisk generering af pladser

Når en administrator opretter en ny forestillingstid, kræver systemet ikke, at vedkommende manuelt opretter 80 individuelle pladsregistreringer. I stedet genererer tjenestefunktionen createShowtime automatisk det komplette sædeoversigt. Den opretter 8 rækker (mærket A til H) med 10 pladser hver, i alt 80 pladser. Hvert sæde tildeles et rækkebogstav, et sædenummer, markeres som ledigt og knyttes til forestillingen. Denne automatisering sparer tid og sikrer konsistens på tværs af alle forestillinger.

7.2 Kaskadesletning

Databaseskemaet bruger i vid udstrækning ON DELETE CASCADE-begrænsninger for at opretholde referentiel integritet. Når en film slettes, fjernes alle dens forestillinger automatisk. Når en forestilling slettes, fjernes alle dens sæder. Når en reservation slettes, fjernes alle dens booked_seats-poster. Denne kaskadeadfærd forhindrer

forældreløse poster og sikrer, at databasen forbliver konsistent uden behov for kompleks oprydningskode.

7.3 Reservationsworkflow

Reservationsprocessen følger et omhyggeligt designet workflow. Først vælger brugeren en film, hvorefter dens forestillinger indlæses. Når en forestilling vælges, oprettes en ventende reservationspost, og sædeoversigten indlæses. Når brugeren vælger pladser, føjes de til den ventende reservation via tabellen booked_seats, og deres tilgængelighedsflag opdateres for at forhindre andre brugere i at vælge dem. Når brugeren bekræfter, beregner systemet den samlede pris, opdaterer reservationsstatus til bekræftet og viser en bekræftelsesmeddelelse. Hvis brugeren navigerer væk uden at bekræfte, forbliver den ventende reservation, men kan ryddes op senere ved hjælp af adminfunktioner.

7.4 Administrationspanel

Administrationspanelet giver omfattende administrationsmuligheder via en fanebaseret grænseflade. Administratorer kan skifte mellem administration af film, forestillinger, bookinger og visning af statistikker. Hver fane indlæser relevante data fra API'en og viser dem i tabeller med indbyggede redigerings- og slettefunktioner. Fanen med statistikker foretager flere API-kald for at indsamle tal for film, forestillinger, bookinger og samlet omsætning og viser dem derefter i visuelle kort for hurtig oversigt.

8. Fremtidige forbedringer

Selvom det nuværende system er fuldt funktionelt, kunne flere forbedringer gøre det endnu bedre. Implementering af korrekt password-hashing ved hjælp af bcrypt ville sikre brugeroplysninger. Tilføjelse af e-mail-bekræftelse for bookinger ville give brugerne kvitteringer. Implementering af en betalingsbehandlingsintegration ville muliggøre reelle køb i stedet for kun reservationer. Tilføjelse af søge- og filterfunktioner ville hjælpe brugerne med at finde specifikke film eller spilletider. Implementering af prisniveauer for pladser (premium-pladser koster mere) ville øge indtægtsmulighederne. Tilføjelse af forbedringer af mobilresponsivitet ville forbedre oplevelsen på telefoner og tablets. Implementering af realtidsmeddelelser, når pladser bliver ledige, ville forbedre brugeroplevelsen. Tilføjelse af muligheden for at annullere bookinger direkte fra brugergrænsefladen i stedet for at kræve administrativ indgriben ville øge brugernes autonomi.

9. Konklusion

Cinema Booking System implementerer med succes en moderne, databasedrevet webapplikation, der løser virkelige problemer i biografbranchen. Gennem omhyggelig kravanalyse, gennemtænkt databasedesign efter tredje normalformsprincipper og ren trelagsarkitektur giver systemet et robust fundament for administration af filmbookinger. Implementeringen demonstrerer bedste praksis inden for webudvikling, herunder korrekt adskillelse af opgaver, rollebaseret adgangskontrol og transaktionsstyring for dataintegritet. Selvom systemet er bygget som et uddannelsesprojekt, kan systemets arkitektur og datamodel skaleres til at understøtte en rigtig biografs drift med passende sikkerhedsforstærkning og infrastrukturforbedringer.

Link til projektet:

https://github.com/NorthSideXD/Databases-JS.git

Note: Du skal lave et account, det kræver ikke et faktuel email address men bare nogen med @ ind i det. Der er også et admin account.

Brugernavn: admin
Password: admin123